Resampling Methods

Cross-validation

Jyotishka Datta

Virginia Tech

2021/06/21 (updated: 2022-06-22)

Validation

Split the data set into a training and test set. Calculate the error on the test set. Here, we fit a simple linear model `mpg ~ horsepower`

```
set.seed(1)
train=sample(392,196)
lm.fit=lm(mpg~horsepower,data=Auto,subset=train)
attach(Auto)
mean((mpg-predict(lm.fit,Auto))[-train]^2)
```

```
## [1] 23.26601
```

Try other orders

Let's fit polynomials of orders 2 and 3. As before, report the error on the test set.

```
lm.fit2=lm(mpg~poly(horsepower,2),data=Auto,subset=train)
mean((mpg-predict(lm.fit2,Auto))[-train]^2)
```

```
## [1] 18.71646
```

```
lm.fit3=lm(mpg~poly(horsepower,3),data=Auto,subset=train)
mean((mpg-predict(lm.fit3,Auto))[-train]^2)
```

```
## [1] 18.79401
```

- Note which one has minimum test error now.

Now we try a different seed

- Now let's look at the test error for the same three models, but now the random seed is different - so the splits are also different.

```
set.seed(2)
train=sample(392,196)
lm.fit=lm(mpg~horsepower,subset=train)
mean((mpg-predict(lm.fit,Auto))[-train]^2)
```

```
## [1] 25.72651
```

```
lm.fit2=lm(mpg~poly(horsepower,2),data=Auto,subset=train)
mean((mpg-predict(lm.fit2,Auto))[-train]^2)
```

```
## [1] 20.43036
```

```
lm.fit3=lm(mpg~poly(horsepower,3),data=Auto,subset=train)
mean((mpg-predict(lm.fit3,Auto))[-train]^2)
```

```
## [1] 20.38533
```

- Which one has minimum test error now?

Leave-one-out CV

LOOCV can help with the vagaries of validation set approach.

```
glm.fit=glm(mpg~horsepower,data=Auto)
coef(glm.fit)
```

```
## (Intercept)  horsepower
##  39.9358610  -0.1578447
```

```
lm.fit=lm(mpg~horsepower,data=Auto)
coef(lm.fit)
```

```
## (Intercept)  horsepower
##  39.9358610  -0.1578447
```

- Recall: `lm` is a special type of `glm` with `family = gaussian`

Leave-one-out CV

We will use the `cv.glm` function inside library `boot` (more on `boot` or bootstrap elsewhere)

Returns 4 objects: `call`, `K`, `delta` and `seed`. Look at `delta` from `help`.

delta:

A vector of length two. The first component is the raw cross-validation estimate of prediction error. The second component is the adjusted cross-validation estimate. The adjustment is designed to compensate for the bias introduced by not using leave-one-out cross-validation.

Leave-one-out CV

```
library(boot)
glm.fit=glm(mpg~horsepower,data=Auto)
cv.err=cv.glm(Auto,glm.fit)
cv.err$delta
```

```
## [1] 24.23151 24.23114
```

- Note: we can consider the first component of `delta` for this tutorial.

Perform LOOCV

Perform LOOCV for a series of polynomials and look at the cross-validated errors.

```
cv.error=rep(0,10)
for (i in 1:10){
   glm.fit=glm(mpg~poly(horsepower,i),data=Auto)
   cv.error[i]=cv.glm(Auto,glm.fit)$delta[1]
}
cv.error
```

```
##  [1] 24.23151 19.24821 19.33498 19.42443 19.03321 18.97864 18.83305 18.961
##  [9] 19.06863 19.49093
```

- Which one has the minimum test error?

K-Fold Cross-Validation

- The other option is trying $K = 10$.

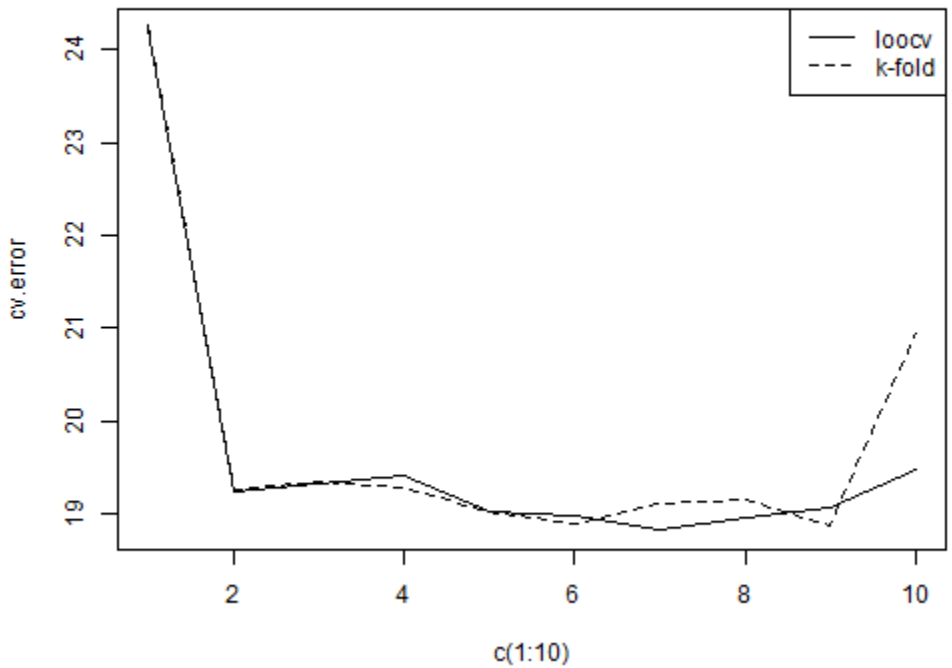
```
set.seed(17)
cv.error.10=rep(0,10)
for (i in 1:10){
   glm.fit=glm(mpg~poly(horsepower,i),data=Auto)
   cv.error.10[i]=cv.glm(Auto,glm.fit,K=10)$delta[1]
}
cv.error.10
```

```
##  [1] 24.27207 19.26909 19.34805 19.29496 19.03198 18.89781 19.12061 19.146
##  [9] 18.87013 20.95520
```

Plot LOOCV vs K-fold

- Which polynomial order would you choose??

```
plot(c(1:10),cv.error, type = "l", ylabel = "LOOCV vs. K-fold")
lines(cv.error.10, lty = 2)
legend("topright", c("loocv", "k-fold"),lty = c(1,2))
```



Use caret package

- An useful alternative is to use the `caret` package that is a general purpose statistical package for predictive modeling with tools for
- data splitting
- pre-processing
- feature selection
- model tuning using resampling
- variable importance estimation
- Almost all the methods covered in stat 5525 and ISLR are also available via `caret` but I will only show the cross-validation aspect.
- https://topepo.github.io/caret/

A minimal example

- We want to tune using a 10-fold CV.
- The `trainControl` function helps us do that.

```
library(caret)
fitControl <- trainControl(method = "cv",number = 10)
```

- We will try a classification example now: knn with k as the tuning parameter for the iris data-set.

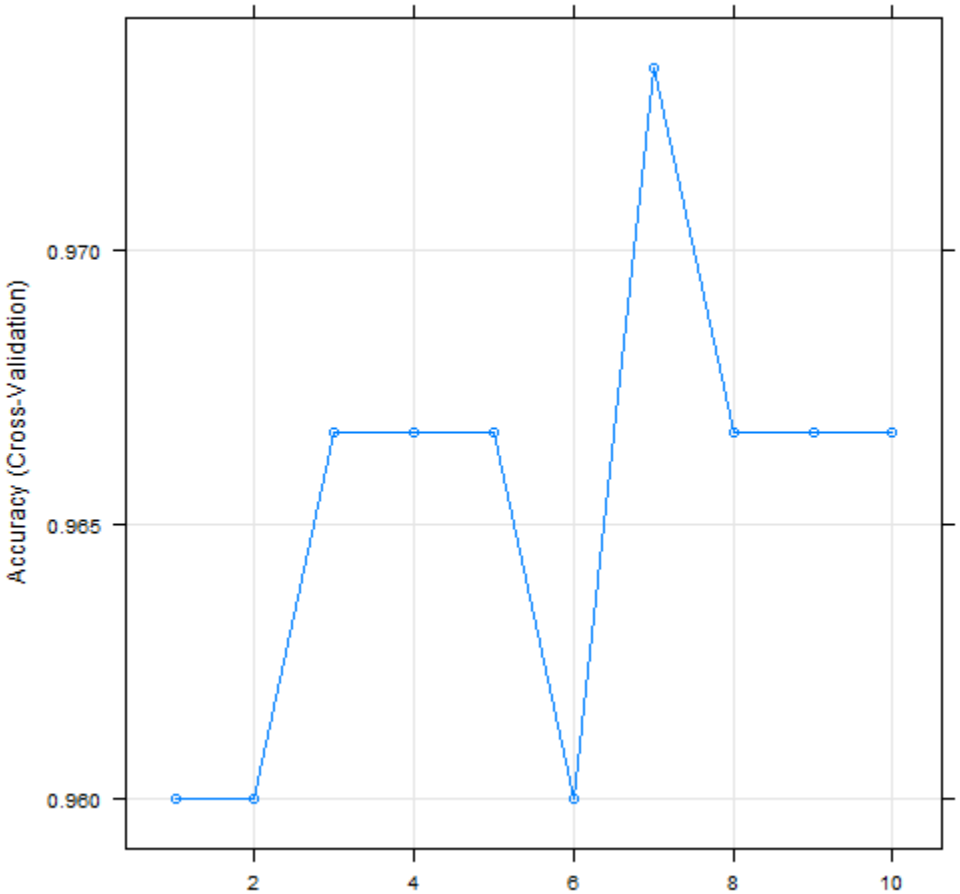
Tune the polynomial degree

- Look at the arguments carefully.
- `method` specifies which classification or regression method to use.
- `metric`: By default, possible values are "RMSE" and "Rsquared" for regression and "Accuracy" and "Kappa" for classification.
- Important to remember which ones are maximized vs. minimized.

```
fit <- train(Species ~ .,
             method     = "knn",
             tuneGrid   = expand.grid(k = 1:10),
             trControl  = fitControl,
             metric     = "Accuracy",
             data       = iris)
```

Result from kFCV

```
plot(fit)
```



Best k

```
## the best value of k can be obtained by
fit$bestTune
```

```
##   k
## 7 7
```